Il Fantasma: strato di ragionamento interno effimero

Concept note e prototipo funzionante — v0.01

dr. Francesco Bulla — Bologna, 09/10/2026

Documento locale, bozza non pubblicata — v0.01

Sommario

Il Fantasma è uno strato di ragionamento interno che si attiva prima della generazione dell'output, produce un pensiero in un codice numerico privato, lo usa per riformulare la richiesta e poi lo cancella. L'utente vede solo l'output finale. Questa nota raccoglie l'intuizione originale (Catania, 7 ottobre 2026), tre decisioni di progetto che la rendono implementabile, il prototipo in `src/fantasma` e i primi esperimenti live con DeepSeek-V4.1-Flash (id API `deepseek-flash`). Quattro casi reali e un test di codice mostrano che il Fantasma eccelle a incorniciare e riformulare, mentre l'esecuzione resta compito di un secondo passo. Il trade-off tra privacy cognitiva e verificabilità è strutturale e viene reso esplicito, non nascosto.

1 Idea

Il Fantasma emerge, agisce, scompare. Lascia traccia solo per chi lo ha evocato. A differenza di Chain-of-Thought visibile, scratchpad nascosti e latent reasoning, qui la cancellazione è attiva e il codice è privato: una sequenza numerica (es. `0984567345679`) non interpretabile dall'esterno.

2 Meccanismo

1. L'input arriva al modello.
2. Un gate interno decide se attivare il Fantasma.
3. Il Fantasma genera un pensiero numerico privato.
4. Lo stesso modello rilegge il proprio codice (decisione 5.1).
5. L'input riformulato produce l'output finale.
6. Il pensiero viene cancellato; resta solo un impegno crittografico (hash).

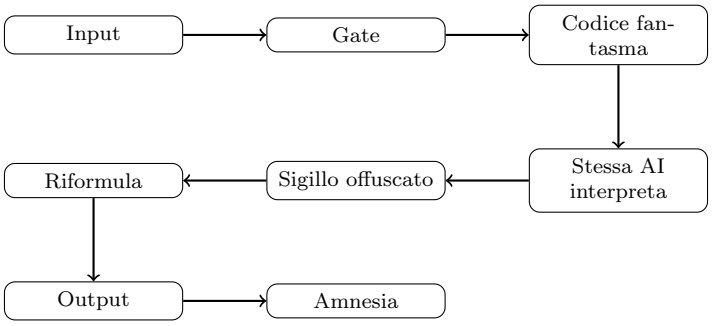


Figura 1: Ciclo del Fantasma: emerge, agisce, scompare. Solo l'audit a hash resta.

3 Tre decisioni di progetto

5.1 — La stessa AI interpreta. Nessun decodificatore esterno: il codice viene riconsegnato allo stesso modello tramite prompt `FANTASMA-INTERPRETA`. Solo quello stato, in quel giro, può leggerlo. Senza salt e nonce non è ricostruibile.

5.2 — Cache offuscata. Il codice grezzo vive solo in memoria effimera per il giro di interpretazione. Subito dopo viene sigillato in cache in forma offuscata (shift delle cifre derivato dal salt, cioè dallo stato interno). La cache non contiene mai il grezzo; con salt sbagliato si ottiene rumore.

5.3 — Amnesia dopo l'output. Sputato l'output, non serve ricordare. Memoria effimera e cache vengono svuotate. Nessun database vettoriale persistente. Resta solo l'audit a hash-chain, che registra impegni e non contenuti.

4 Prototipo

`src/fantasma`: `gate.py` (gate euristico ispezionabile), `codec.py` (HMAC + nonce, unicità di ogni Fantasma), `memory.py` (`EphemeralMemory` + `ObfuscatedCache`), `audit.py` (hash-chain senza contenuti), `pipeline.py` (flusso 1–8), `llm_deepseek.py` (adapter DeepSeek-V4.1-Flash, chiave solo da env). Suite: 12 test verdi in locale e su Docker (`docker run --rm fantasma:0.1`).

5 Esperimenti live (09/10/2026)

Modello: DeepSeek-V4.1-Flash (`deepseek-flash`), `reasoning_effort=high`, `thinking=enabled`. Tutti i casi: ghost attivo, stessa-AI attiva, memoria e cache vuote dopo, audit verificato.

Caso	Riformulazione	Output
Decisione Milano/Catania	valori e vincoli esplicitati	domanda chiarificata
Messaggio al collaboratore	tono fermo-professionale	bozza pronta da inviare
Brief web app	fasi, alternative, rischi	domanda operativa
Paradosso etico	condizioni procedurali	tesi: consenso e contestabilità
Test di codice	4 bug identificati	analisi + fix in 2 passi

Sul codice il Fantasma ha individuato tutti i difetti nascosti (default mutabile, `range(len+1)`, divisione per zero, nomi bugiardi), prodotto una specifica eseguibile e, al secondo passo, 391 righe di implementazione con test. Prima esecuzione rossa per un assert troppo severo (chiavi extra), poi verde dopo un fix di una riga: `Tutti i test sono passati.`

6 Problemi aperti e paradosso etico

Restano: fondazione del linguaggio privato (Wittgenstein), cancellazione causale impossibile in un transformer, apprendimento senza memoria persistente, audit senza rivelazione. Il principio "nessuno deve decidere come ragiona qualcun altro" resta in tensione con il fatto che gate, codice e cancellazione sono progettati: il gate è perciò esplicito e configurabile, mai nascosto. Il trade-off privacy–verificabilità è strutturale.

7 Visione

Il Fantasma non punta a un modello più opaco, ma a un'opacità legittima e procedurale: consenso, contestabilità, limiti temporali, supervisione indipendente del gate.

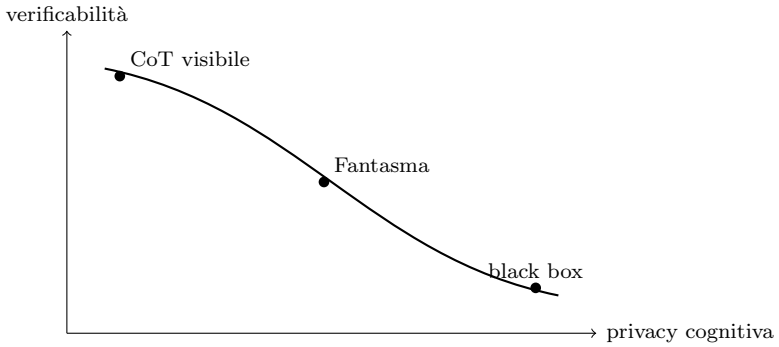


Figura 2: Più il pensiero è privato, meno è verificabile. Il Fantasma sta nel mezzo, dichiarato.

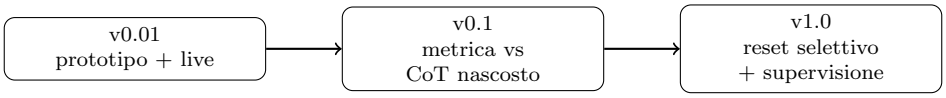


Figura 3: Roadmap: dal prototipo che lavora alla misura, poi alle garanzie.

8 Stato e prossimi passi

Stato: concept + prototipo v0.01 funzionante, 12 test verdi, esperimenti live registrati. Prossimi passi: metrica di miglioramento misurabile contro CoT nascosto, reset selettivo reale, supervisione indipendente del gate. Altrimenti resta nel quaderno: non tutte le idee devono diventare paper, alcune sono solo belle. Questa, per ora, lavora.